

Titanium Sponsors

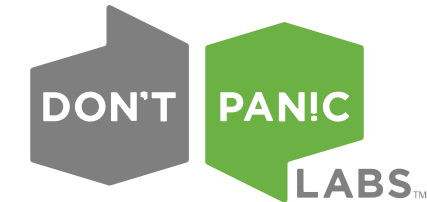


Platinum

Sponsors

Gold

Sponsors



Friends of KCDC



Other Awesome Organizations



The incident, three ways

Lock-wait timeout — row contention

Runtime error — an implicit data contract violated

Timestamp gaps — slow queries

The incident, three ways

Lock-wait timeout — row contention

Runtime error — an implicit data contract violated

Timestamp gaps — slow queries

All three lead to the same result: a potential customer facing outage

AI Assisted Legacy Refactors

Brad Boggs

One application

At the epicenter of nearly every on-call incident.

```
1  @~ $__ = [#..]; %%(&$_)
2  %%fn ($_, &~) {
3  $__ ->> ~[#] :: <<0#>>;
4  ~~ = <<0#>> ?: !!;
5  } @@ #~;
6  $$$ .= @{~}; ->> $~ ;;
7  #~ <<= ???; █
```



```
SELECT * FROM ...  
WHERE call_reason = 12
```

```
@~ $__ = [#..];  
%%fn ($_, &~) {  
  $__ ->> ~[#];  
  ~~ = <<0#>>;  
} ?: !!;  
$## .= @{~};  
~~ <<= ???;
```

```
if (~$_) {  
  ## << "...";  
}  
$~ = %%(#$_);  
->> $~ ;;  
} @@ #~;  
@~ :: !!;
```



A rewrite was attempted.
And aborted.

Two blockers

01

A third-party library detail caused a performance regression.

Specific. Manageable. A fix was found later, and left unimplemented.

“We weren't 100% confident in how we were going to test and approve the new version.”

Two blockers

01

A third-party library detail caused a performance regression.

Specific. Manageable. A fix was found later, and left unimplemented.

02

No way to test and approve the new version.

Which is why the first one was never fixed.

Why legacy resists change

No understanding of the system

Why legacy resists change

No understanding of the system

No tests to protect us in its absence

Why legacy resists change

No understanding of the system

No tests to protect us in its absence

Real business cost when we get it wrong

2004

“to me, legacy code is simply code
without tests”

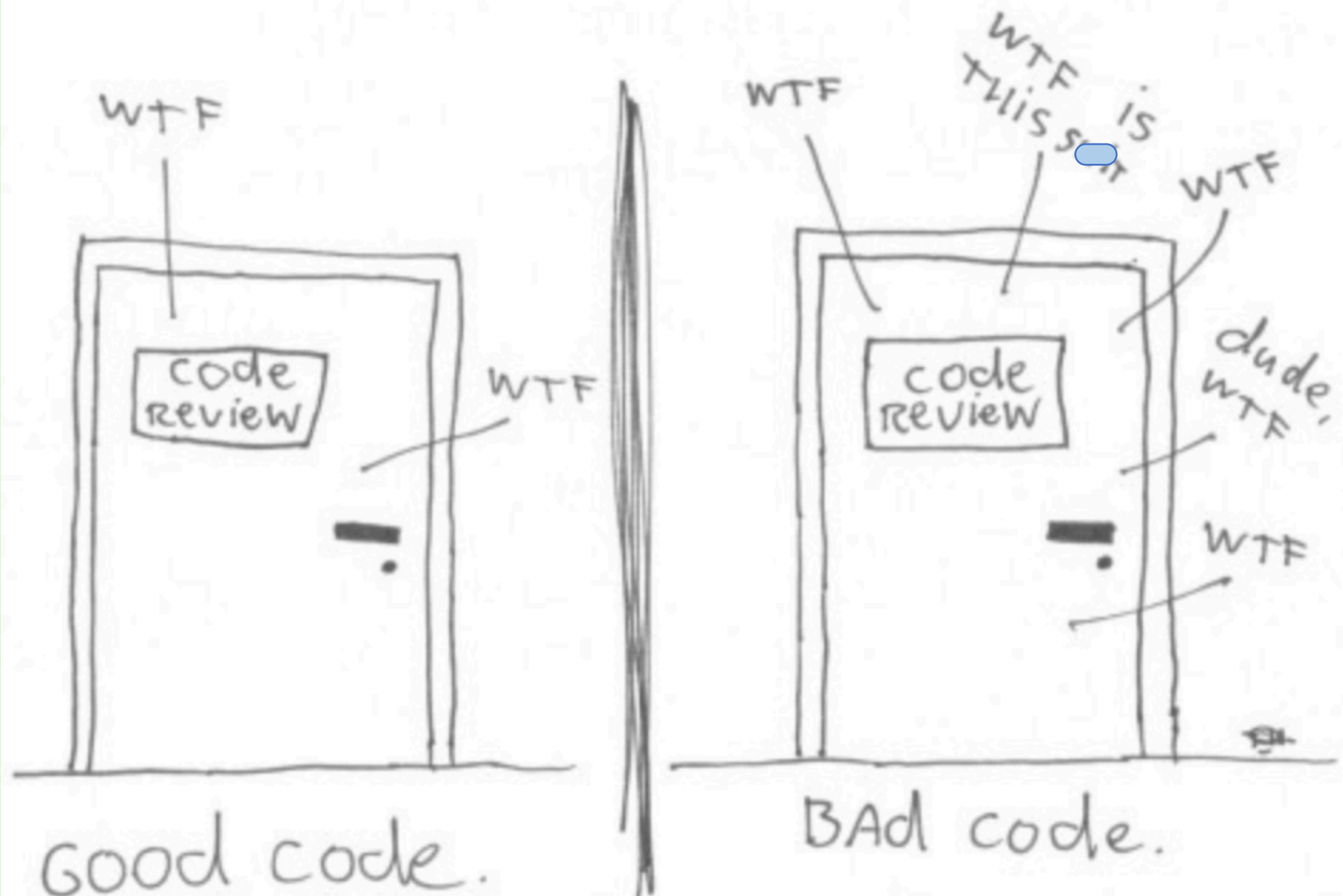
Michael Feathers

2022

“Legacy code is code that delivers more in value than it costs to maintain.”

Brad Ediger

The ONLY VALID MEASUREMENT OF CODE QUALITY: WTFs/MINUTE



“We weren't 100% confident in how we were going to test and approve the new version.”

technical specification

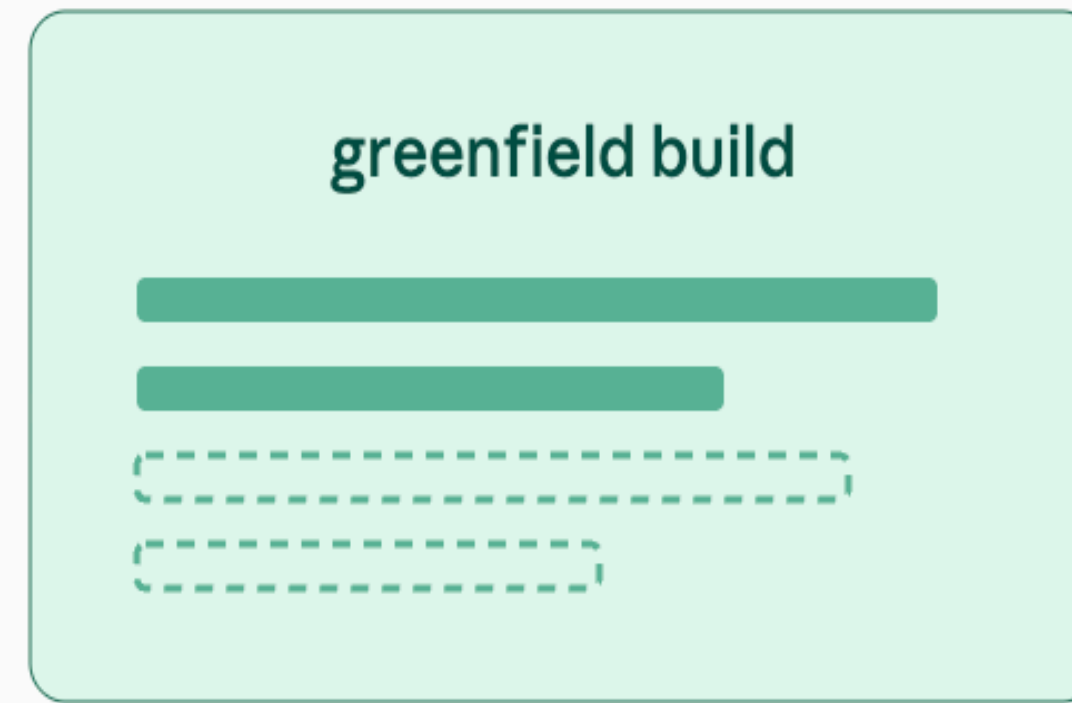
Translates approved needs into an implementable and verifiable description of a system.

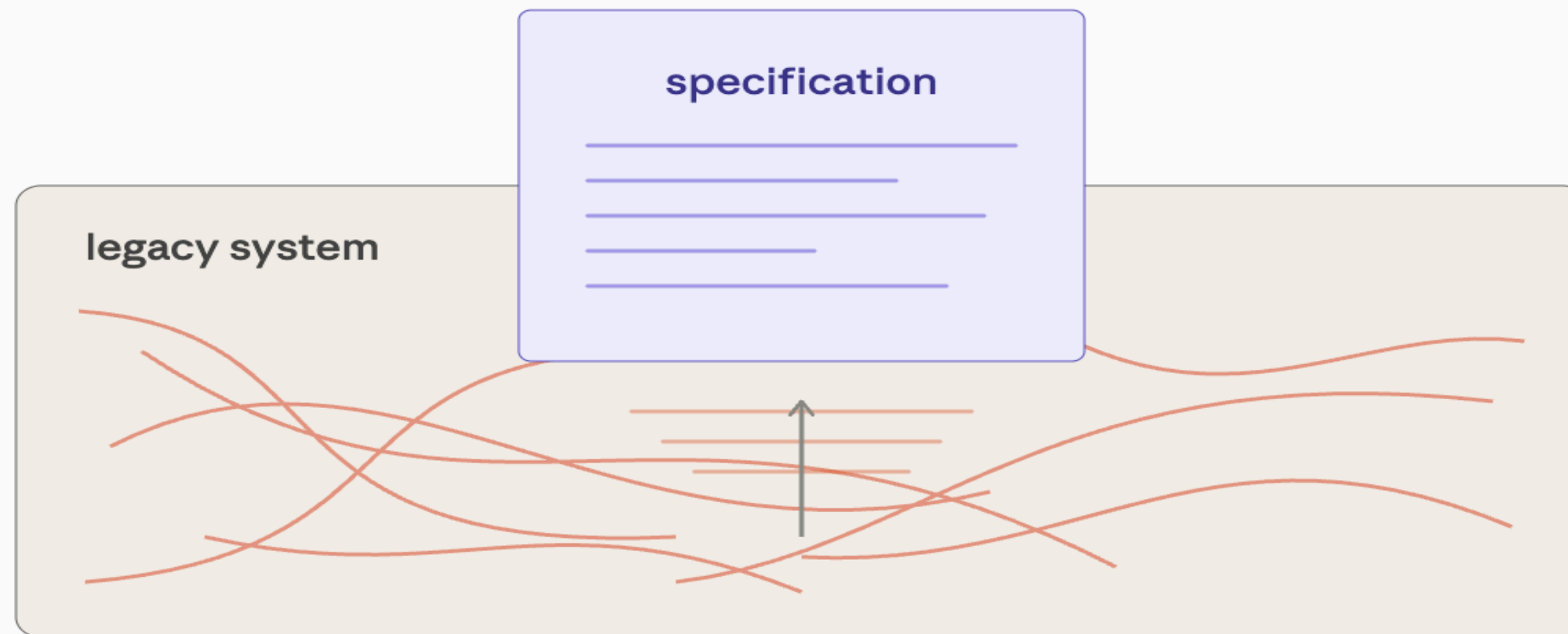
Interfaces. Data. Constraints. Engineering decisions.



written before the code

built against





uncovered from within

The definition of what should
be collapses to a subset of
what is.

“With a sufficient number of users of an API, it does not matter what you promise in the contract: all observable behaviors of your system will be depended on by somebody.”

legacy system specification

What must remain true at the edges no matter what else changes

On Call Incident # 2

One method, measured

1,489 lines in one method

92% of it covered by a single transaction

1 synchronous HTTP call inside that transaction

23 writes

4 reads

464 lines, median distance from writing a value to reading it back

48 local variables live at once at the deepest point

2–6

chunks in human short-term memory

48

local variables live at once

Lack of knowledge · lack of information · lack of processing power

Felienne Hermans, *The Programmer's Brain*, 2021

A context window problem

Context window is the number of tokens the model can process at once. It's like a buffer that stores the input sequence. The model can only process tokens within this window. If the input sequence is longer than the context window, the model will only process the most recent tokens, and the earlier tokens will be forgotten. This is a problem because the model needs to know the entire input sequence to generate the correct output. For example, if the context window is 10 tokens and the input sequence is 15 tokens long, the model will only process the last 10 tokens and will not know the first 5 tokens. This can lead to incorrect outputs.

A probabilistic tool is a bad way to produce something you have to trust.

It is a very good way to produce a candidate you are going to check .

Planning for the remodel is greenlit.

With a clear path to testing and approving what replaces it.

Workflow Demo

CHARACTERIZATION TESTS

They pin current behavior as-is.

They make no claim that it is correct.

On their own they pin everything equally. The edges tell you which of it counts.

Find the edges. Generate characterization tests at the edges. Generate and evaluate the spec from the characterization tests, and finally make the change.

Module · whole app · interconnected systems

G. K. Chesterton



“There exists in such a case a certain institution or law; let us say, for the sake of simplicity, a fence or gate erected across a road. The more modern type of reformer goes gaily up to it and says, ‘I don't see the use of this; let us clear it away.’ To which the more intelligent type of reformer will do well to answer: ‘If you don't see the use of it, I certainly won't let you clear it away. Go away and think. Then, when you can come back and tell me that you do see the use of it, I may allow you to destroy it.’”

G. K. Chesterton, *The Thing*, 1929

“We weren't 100% confident in how we were going to test and approve the new version.”

legacy system specification

What must remain true at the edges no matter what else changes

Provide Feedback on the
talk



Talk materials Github
Repo



Part-time consulting available

